2048 as a Markov Decision Problem

Problem formulation — Dynamic Programming and Reinforcement Learning, project work

Name Surname (matricola) Name Surname (matricola)

November 11, 2026

1 Motivation

2048 (G. Cirulli, 2014 [4]) is a single-player puzzle played on a 4×4 grid of numbered tiles. Every move is a deterministic slide of all the tiles followed by the random appearance of a new tile, so the game is a Markov decision problem whose model is completely known. Small versions of the game (fewer cells, a smaller target tile) can be solved exactly by dynamic programming. The original game cannot: its state space is far too large. The project is built on this gap.

- **DP part.** On small boards, compute the optimal policy exactly and use it to answer questions that are hard to settle by playing: how well can the game be played, how good are the rules of thumb used by human players, and how much of the outcome is luck.
- **RL part.** Remove the knowledge of the model and move to the original 4×4 board, where only learning with function approximation is feasible. The exact solutions of the DP part validate the learning algorithms.

2 The game

The game is played on a board of $R \times C$ cells with a target tile 2^K. In the original game $R = C = 4$, $K = 11$ (tile 2048) and $p_4 = 0.1$.

1. A move $u \in \{\mathsf{down}, \mathsf{left}, \mathsf{right}, \mathsf{up}\}$ slides all the tiles as far as possible in that direction. Two tiles with the same value that collide merge into one tile carrying their sum. A tile created by a merge cannot merge again in the same move, and in each line the merges are resolved starting from the side the tiles move to: moving left, 2 2 2 2 becomes 4 4. The score increases by the value of every tile created by a merge.
2. A move is legal only if it changes the board.
3. After every legal move a new tile appears in an empty cell chosen uniformly at random: a 2 with probability $1 - p_4$, a 4 with probability p_4. The game starts with two such tiles in two distinct cells.
4. The game is won when a move creates the target tile, and lost when no legal move exists.

3 Formulation as a Markov decision problem

State. A board is encoded by the exponents of its tiles, $x = (x_1, \ldots, x_{RC})$ in row-major order, with $x_i = 0$ for an empty cell and $x_i = k$ for a tile 2^k. The state x_t is the board observed before the t-th move, that is, after the random tile. The initial state x_0 has distribution p_0: two tiles in two distinct cells chosen uniformly at random, each a 2 with probability $1 - p_4$ or a 4 with

probability p_4. Since the game ends as soon as the target is created, $x_i \in \{0, \dots, K-1\}$, and the state space $\mathcal{X}$ is the set of boards reachable from p_0. Two kinds of terminal situations exist: the win, reached when a move creates 2^K, and the boards with no legal move (game over).

Control. The admissible controls are the legal moves,

$$\mathcal{U}(x) = \{u \in \{\mathsf{down}, \mathsf{left}, \mathsf{right}, \mathsf{up}\} : M(x,u) \neq x\},$$

where M is the slide/merge map defined below; the game is over exactly when $\mathcal{U}(x) = \emptyset$.

Deterministic part of a move. $y = M(x,u)$ is the board after the slide/merge and before the new tile, called the *afterstate*. M acts line by line: each row (for left/right) or column (for up/down), read towards the side of the move, is compacted by removing the empty cells, adjacent equal pairs are merged from the front with each tile used at most once, and the line is padded with empty cells. The points of the move, $r(x,u)$, are the sum of the values of the merged tiles.

Disturbance. $w_t = (i_t, v_t)$, where i_t is uniform over the set $E(y_t)$ of empty cells of the afterstate $y_t = M(x_t, u_t)$, and $v_t \in \{1,2\}$ is the exponent of the new tile, with $P(v_t = 1) = 1 - p_4$ and $P(v_t = 2) = p_4$. Given y_t, w_t is independent of the past; its distribution depends on (x_t, u_t) only through y_t.

Dynamics. Writing $y + v\, e_i$ for the board y with exponent v placed in the empty cell i,

$$x_{t+1} = f(x_t, u_t, w_t) = M(x_t, u_t) + v_t\, e_{i_t}.$$

Objective. Maximise the probability of creating the target tile,

$$V^*(x) = \max_{\pi}\ P^{\pi}\big(\text{the target tile is created} \mid x_0 = x\big).$$

In the cost form used in the course [1], this is a problem with no stage cost and a terminal cost equal to 1 for a lost game and 0 for a won game; its optimal cost is $J^*(x) = 1 - V^*(x)$, the minimum probability of losing. A second objective is studied for comparison: the expected score, with stage reward $r(x_t, u_t)$ and no stop at the target tile.

Bellman equation. With the afterstate value W^*,

$$V^*(x) = \begin{cases} 0 & \text{if } \mathcal{U}(x) = \emptyset, \\ \max\limits_{u \in \mathcal{U}(x)} W^*\big(M(x,u)\big) & \text{otherwise,} \end{cases} \tag{1}$$

$$W^*(y) = \begin{cases} 1 & \text{if } \max_i y_i = K, \\ \displaystyle\sum_{i \in E(y)} \frac{1}{|E(y)|}\Big[(1-p_4)\, V^*(y + e_i) + p_4\, V^*(y + 2e_i)\Big] & \text{otherwise.} \end{cases} \tag{2}$$

For the expected score, the maximisation becomes $\max_u \big[r(x,u) + W(M(x,u))\big]$ and the first case of (2) disappears.

4 Structural properties

Proposition 1 (the tile sum is a clock)**.** *Let $\sigma(x) = \sum_{i:\, x_i > 0} 2^{x_i}$ be the sum of the tiles. For every board x, legal move u and disturbance $w = (i, v)$,*

$$\sigma\big(M(x,u)\big) = \sigma(x), \qquad \sigma\big(f(x,u,w)\big) = \sigma(x) + 2^v \in \{\sigma(x) + 2,\ \sigma(x) + 4\}.$$

Proof. Sliding moves tiles without changing their values, and a merge replaces two tiles 2^k with one tile $2^{k+1} = 2^k + 2^k$. The new tile adds 2^v. □

Corollary 1. *(i) The tile sum strictly increases along every trajectory, so no state is visited twice: the transition graph of the reachable states is acyclic.*

(ii) Before the target is created every tile is at most 2^{K-1}, so $\sigma(x_t) \le RC \cdot 2^{K-1}$. Since σ grows by at least 2 per move, every game ends within $RC \cdot 2^{K-2}$ moves, whatever the policy. All the policies are proper, and (1)–(2) has a unique solution.

(iii) Let $\mathcal{X}_\sigma = \{x \in \mathcal{X} : \sigma(x) = \sigma\}$. Transitions only go from $\mathcal{X}_\sigma$ to $\mathcal{X}_{\sigma+2} \cup \mathcal{X}_{\sigma+4}$. Hence V^*, which value iteration computes in a finite number of iterations, is also computed exactly by a single backward recursion over σ, like the finite-horizon DP algorithm with stage index $\sigma/2$.

Total-cost formulation. In the terms of the course [1], 2048 is an infinite-horizon problem with total cost and two kinds of termination states, the win and the boards without legal moves, made absorbing and cost-free. Corollary 1(ii) gives the assumptions of the total-cost case: termination states exist (A2) and every policy reaches them from every state (A3), here with probability one and within $RC \cdot 2^{K-2}$ moves. The Bellman equation (1)–(2) therefore has a unique solution, and value iteration converges to it from $V^{(0)} = 0$ with no discount; updating the states in decreasing tile sum, one sweep is enough, and this is the backward recursion below.

Remark (afterstates). A move factors into a deterministic, controlled part $x \mapsto y = M(x, u)$ and a random, uncontrolled part $y \mapsto x'$. The expectation over the disturbance can therefore be computed in two steps: first the afterstate y of every move, then the average over the new tile in (2). The same decomposition gives the RL part an exact parametrisation of the action values, $q(x, u) = r(x, u) + V(M(x, u))$.

Remark (symmetries). Rotations and reflections of the board (8 for a square board, 4 for a rectangular one) map the problem onto itself, with the moves mapped accordingly, so V^* is invariant under them. They could reduce the state space by up to that factor; the base implementation does not exploit them and uses them as a correctness test.

5 Exact DP algorithm

Algorithm 1 Exact DP by stages

Forward pass: starting from the initial distribution (two tiles), generate the reachable sets $\mathcal{X}_\sigma$ for increasing σ
for $\sigma = \sigma_{\max},\ \sigma_{\max} - 2,\ \ldots,\ \sigma_{\min}$ **do** ▷ backward pass
  for each $x \in \mathcal{X}_\sigma$ **do**
    compute $W^*(M(x, u))$ for every $u \in \mathcal{U}(x)$ with (2) ▷ uses V^* on $\mathcal{X}_{\sigma+2} \cup \mathcal{X}_{\sigma+4}$
    $V^*(x) \leftarrow \max_u W^*(M(x, u))$, $\mu^*(x) \leftarrow$ a maximising u ▷ $V^*(x) = 0$ if $\mathcal{U}(x) = \emptyset$
  end for
end for
return μ^* and the value at the start $\sum_x p_0(x)\, V^*(x)$

Replacing the maximisation with the action chosen by a fixed policy gives the exact value of that policy (policy evaluation), which is used for the benchmark policies. Ties are broken in the fixed order down, left, right, up. The cost is proportional to $\sum_\sigma |\mathcal{X}_\sigma| \cdot |\mathcal{U}| \cdot 2RC$.

6 Size of the state space

Table 1 reports the number of reachable states, computed with the accompanying code, together with the naive bound K^{RC} (each cell holds an exponent in $\{0, \dots, K-1\}$). Doubling the target roughly doubles the states. Adding cells is far more expensive: with target 64, the 2×4 board has about 30 times the states of the 2×3 board, and the 3×3 board about 180 times. For the original game the bound is $11^{16} \approx 4.6 \cdot 10^{16}$: exact DP is out of reach, which motivates the RL part.

Board	Target	Reachable states $\lvert\mathcal{X}\rvert$	Bound K^{RC}
2×2	16	214	256
2×2	32	422	625
2×3	32	11 222	15 625
2×3	64	27 022	46 656
2×3	128	52 486	117 649
2×4	64	881 260	1 679 616
2×4	128	2 525 754	5 764 801
3×3	64	4 820 069	10 077 696
3×3	128	17 244 751	40 353 607
4×4	2048	intractable	$\approx 4.6 \cdot 10^{16}$

Table 1: Reachable decision states (target not yet created) and naive bound.

7 Benchmark policies

- **Corner rule**: the first legal move in the order down, left, right, up. It is the classic rule of thumb of keeping the large tiles in one corner and pressing up only when forced.
- **Greedy**: the legal move with the largest immediate score $r(x, u)$, ties broken in the same order.
- **Random**: a uniformly random legal move.

8 Aspects to investigate

1. The optimal win probability as a function of board size and target, compared with the benchmark policies.
2. The structure of the optimal policy: in which situations it departs from the corner rule, and which of these departures matter most (expected visits $\times$ loss of win probability).
3. The role of chance: sensitivity of the optimal win probability to p_4, and the distribution of the game length (Monte Carlo).
4. Win probability or expected score: do the two optimal policies differ, and where?
5. The curse of dimensionality: growth of $\lvert\mathcal{X}\rvert$ and of the computing time with the board.
6. Validation: Monte Carlo simulation of the optimal policy against the exact values.
7. Value iteration on the total-cost formulation: number of iterations to the exact fixed point, compared with the backward recursion.

9 From DP to RL

The RL part keeps the game and adds the constraints that make exact DP impossible.

- **Unknown model**: the agent only interacts with a simulator (`reset`/`step`) and does not know the spawn distribution.
- **Original board**: 4×4 with target 2048, where tabular methods are not feasible. The action values are approximated linearly, $\hat{q}(x, u; \mathbf{w}) = r(x, u) + \mathbf{w}^\top \boldsymbol{\phi}(M(x, u))$, with the binary n-tuple features of Szubert and Jaśkowski [3], and learned with episodic semi-gradient Sarsa.
- **Validation**: on the small boards the learned values and policies are compared with the exact V^* and v_π of the DP part.

Methods, all from the course [2]: TD(0), n-step TD and Monte Carlo prediction (§6.1–6.3, §7.1); Monte Carlo control, Sarsa, Q-learning, Expected Sarsa and n-step Sarsa (§5.4, §6.4–6.6, §7.2), also with the action values written as $q(x, u) = r(x, u) + V(M(x, u))$; episodic semi-gradient Sarsa with linear function approximation (§10.1). A deep Q-network (DQN), built as in the PyTorch tutorial, is added as a final test.

References

[1] D. P. Bertsekas, *Reinforcement Learning and Optimal Control*, Athena Scientific, 2019.

[2] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[3] M. Szubert, W. Jaśkowski, "Temporal difference learning of N-tuple networks for the game 2048", *IEEE Conference on Computational Intelligence and Games (CIG)*, 2014.

[4] G. Cirulli, *2048*, 2014. `https://github.com/gabrielecirulli/2048`